SYSTEMS ENGINEERING

Architecture of a Multi-Strategy Trading System

How 30+ independent strategies share one market data connection — without ever blocking each other

> *“Tick data is not a stream to consume. It is a value to look up.” That reframing turned a messaging problem into a shared-state problem — and changed the entire design.*

Dhrumil Savaliya

Trading Systems & Infrastructure

July 2026

At a glance

~40 CONCURRENT PROCESSES	**30+** STRATEGY PROCESSES	**8** EXECUTION ACCOUNTS
1 FEED CONNECTIONS NEEDED	**~1,000** LIVE INSTRUMENTS	**µs** TICK READ LATENCY
21 SIGNAL BUS PORTS	**63** PYTHON SOURCE FILES	**19.5k** LINES OF CODE

CONTENTS

1. What this system does 4

2. The core design problem 4

3. System architecture 5

4. Layer 2 — The shared-memory tick bus 6

5. Layer 3 — The strategy runtime 8

6. Layer 4 — The signal bus 9

7. Layer 5 — Execution adapters 10

8. Concurrency model 11

9. Observability and persistence 12

10. Deployment 13

11. Technology stack 14

12. Scale 15

13. What I would tell someone building this 15

14. Closing 16

1. What this system does

This is the architecture of a production intraday **index options** trading system that I designed and built.

The system solves one specific problem:

> *Run **many independent trading strategies simultaneously**, feed all of them from **one market data connection**, and route every resulting signal to **multiple execution accounts** — without any strategy ever being able to slow down, block, or crash another.*

That single constraint drove every architectural decision that follows.

The result is a system where roughly **40 processes** run concurrently on a single machine, sharing market data through memory rather than the network, and communicating through a message bus rather than function calls.

---

2. The core design problem

A naive multi-strategy system is a single process with a loop:

```
for strategy in strategies:
    strategy.on_tick(tick)
```

This breaks down fast, in three ways.

One slow strategy blocks all of them. If one strategy does a heavy numerical calculation on a tick, every other strategy waits. In a market where entries are timed to the second, that is unacceptable.

One crash kills all of them. An unhandled exception in any strategy takes down the entire process — including strategies that currently hold open positions.

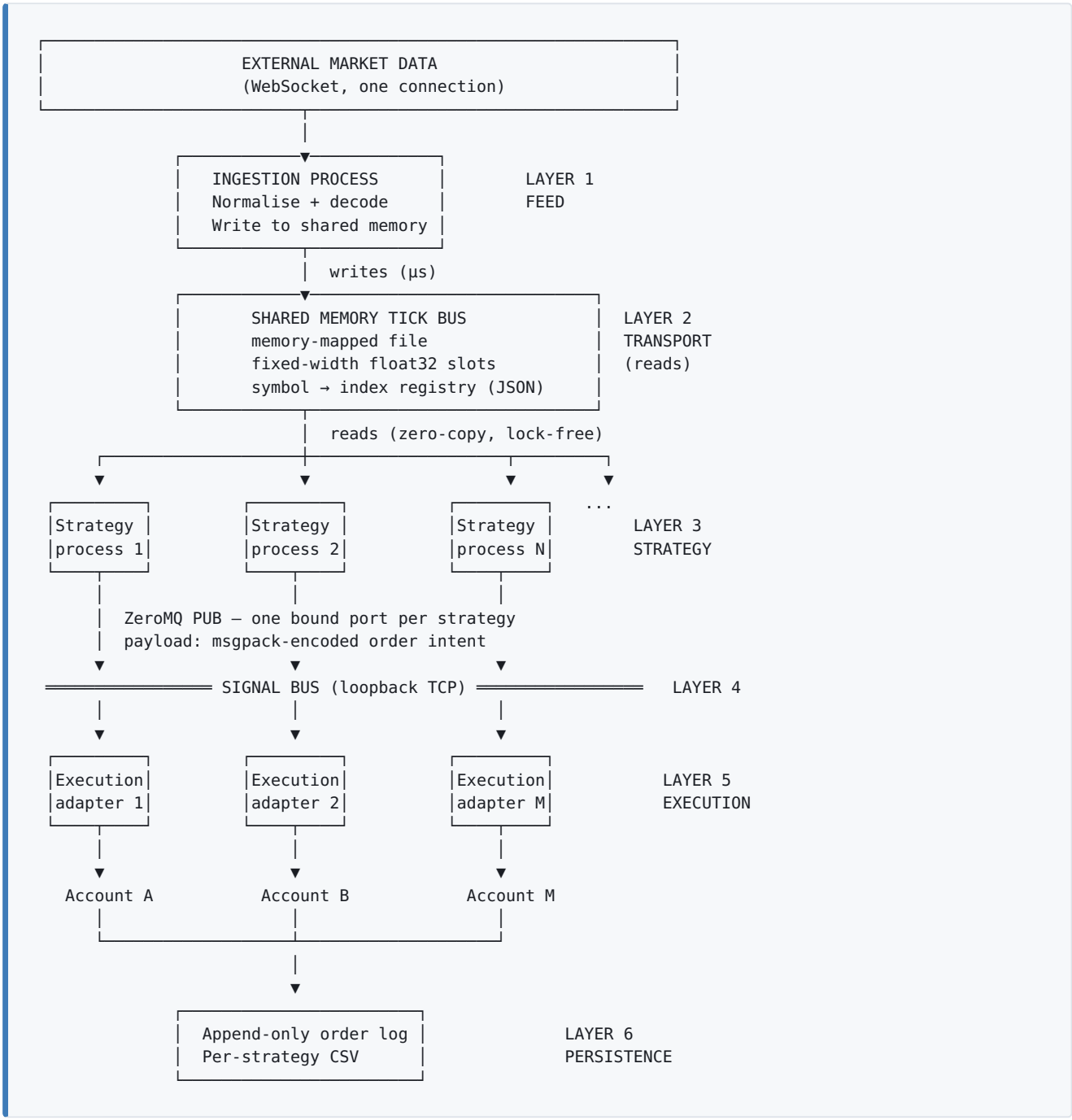
Deployment becomes all-or-nothing. You cannot restart, patch, or add a strategy without stopping everything.

The alternative — one process per strategy — solves all three, but creates a new problem: **how do 30+ separate processes read the same market data without each opening its own connection?**

That question is where the interesting engineering lives.

---

3. System architecture



Six layers. Each one is a process boundary, and each boundary was chosen deliberately.

4. Layer 2 — The shared-memory tick bus

This is the piece I am most satisfied with, so it gets its own section.

The problem

30+ processes need the latest price for ~1,000 instruments, refreshed continuously. The obvious options are all bad:

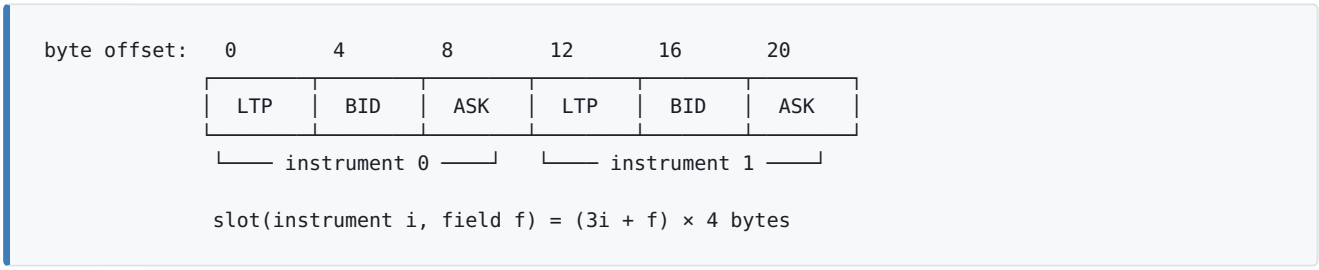
Approach	Why it fails here
Each process opens its own feed connection	Vendor connection limits; 30× bandwidth; 30× the cost
Feed process publishes ticks over the message bus	Every tick fans out to 30 subscribers — enormous message volume for data that is overwritten milliseconds later
Shared Redis / in-memory DB	Network hop + serialise + deserialise on every single read
Write ticks to a database	Orders of magnitude too slow

The insight is that **tick data is not a stream to consume — it is a value to look up.** No strategy cares about the tick from 400 ms ago. It only ever wants *the current price, right now*.

That turns a messaging problem into a **shared state problem**, and shared state has a much cheaper solution.

The design

One memory-mapped file. Fixed-width slots. Three `float32` values per instrument:



A separate JSON registry maps `symbol → index`, written once at startup when the instrument universe is resolved.

Writer (the ingestion process):

```
mmapped_file.seek(index * 4)
mmapped_file.write(struct.pack('f', value))
```

Reader (every strategy process):

```
def read_value(index):
    mmapped_file.seek(3 * index * 4)
    return struct.unpack('f', mmapped_file.read(4))[0]
```

That is the entire interface. Two functions.

Why this works

Zero copy. The OS page cache backs the file. Reader processes map the same physical pages. A read is a pointer dereference, not a syscall, not a network round-trip.

Lock-free on the read path. A `float32` write to an aligned address is atomic on x86-64. A reader either sees the old price or the new price — never a torn half-written value. Since strategies always want the newest value anyway, there is nothing to coordinate. No mutex, no reader contention, no lock convoy at market open.

Constant cost as strategies scale. Adding the 31st strategy adds exactly zero load to the ingestion process. It just maps the same file. This is the property that makes the whole architecture viable.

Naturally self-cleaning. There is no queue to drain, no backlog to manage, no high-water mark to tune. Stale data overwrites itself by definition.

The measured read latency sits in the **microsecond range** — several orders of magnitude below any network-based alternative, and small enough that it is not a factor in strategy timing at all.

The trade-off I accepted

Shared memory gives you speed by giving up structure. There is no schema, no type safety, no versioning. The layout — stride, field order, offset arithmetic — is an **implicit contract** between writer and reader.

If a reader assumes the wrong stride, it does not throw an error. It silently returns a number from the wrong instrument.

I contained that risk by treating the layout as a versioned interface: separate files per layout version, and readers pinned explicitly to the version they were written against. It is a discipline, not a guarantee — which is the honest trade for this much performance.

---

5. Layer 3 — The strategy runtime

Every strategy is an **independent OS process** with a uniform lifecycle:

```
1.  Map the tick bus (read-only)
2.  Load the symbol → index registry
3.  Bind a ZeroMQ PUB socket on its own dedicated port
4.  Load the day's instrument universe
5.  Run its own loop  ──────────────┐
6.  Publish order intents           │  fully isolated
7.  Write its own trade log         │
8.  Exit at session close  ─────────┘
```

Steps 1–4 and 6–8 are shared infrastructure. Step 5 is the only part that differs between strategies — and it is the only part that is proprietary.

That separation is the point. **The framework is generic; the alpha is one function.**

Process isolation, concretely

Because every strategy is its own process:

- A crash is contained to one strategy. The other 29 never notice.
- CPU-heavy work is scheduled by the OS across cores automatically — no GIL contention, no async starvation, no cooperative-yielding discipline required.
- Any strategy can be restarted mid-session without touching the rest.
- Strategies can run on **different Python versions** in the same system. I use this in production: one component runs on a different interpreter version than everything else because of a third-party SDK constraint. In a monolith, that single dependency would have dictated the version for the entire codebase.

That last point is underrated. Process boundaries are also **dependency boundaries**.

The publish contract

Every strategy emits the same message shape. This is the only thing execution adapters know about strategies:

```
{
    "instrument_token": token,      # exchange identifier
    "qty":              lots,       # in lots, not units
    "side":             "BUY" | "SELL",
    "segment":          segment,
    "order_type":       order_type,
    "order_number":     seq,        # per-strategy sequence
    "price":            reference_price,
    "leg_type":         leg_role,
    "symbol":           symbol,
}
```

Serialised with **msgpack** — roughly 30–50% smaller than JSON and meaningfully faster to encode and decode. On a hot path that fires during volatile market moves, that matters.

One deliberate choice: **quantity travels in lots, not units.** Lot size is an exchange-level property that changes over time, and each account may size differently. Keeping the signal in abstract lots means a lot-size change never requires touching strategy code — it is a single constant in the execution layer.

---

6. Layer 4 — The signal bus

Strategies and execution adapters are connected by **ZeroMQ PUB/SUB over loopback TCP**, with one bound port per strategy, allocated from a central port registry.

```
Strategy  ──PUB:port──┐
                      ├──→ Adapter A   (subscribes to a subset)
Strategy  ──PUB:port──┤
                      ├──→ Adapter B   (subscribes to a different subset)
Strategy  ──PUB:port──┘
```

Why PUB/SUB rather than direct calls

Fan-out is free. One strategy publishes once. Any number of adapters receive it. Adding a new execution account means starting a new subscriber process — no strategy is modified, restarted, or even aware of it.

Subscription is configuration, not code. A single config file declares which adapter listens to which strategies, and at what size multiplier. Changing capital allocation across accounts is a config edit, not a deployment.

Publishers never block. ZeroMQ PUB is asynchronous and non-blocking by design. If a subscriber is slow or absent, the publisher is entirely unaffected. A strategy's timing can

never be perturbed by execution-side latency — which is exactly the isolation guarantee I needed.

Topic filtering is built in. Multipart messages carry a topic frame, so subscribers filter at the socket layer rather than in application code.

The trade-off I accepted

PUB/SUB is fire-and-forget. There is no delivery acknowledgement. A subscriber that is down when a message is published does not receive it later.

I handled this at the **execution layer** rather than the transport layer, because that is where the right primitive lives — see the next section.

---

7. Layer 5 — Execution adapters

Each execution account gets its own adapter process. An adapter:

1. Reads config to learn which strategies it follows and at what multiplier
2. Subscribes to exactly those ports
3. Translates lots into units for its account
4. Submits the order through that account's API
5. Logs the result

The adapters are where the system's diversity lives — different accounts expose genuinely different APIs, with different auth flows, order semantics, and latency profiles.

Delta orders vs. target positions

The most important distinction between adapters:

Model	Message meaning	Behaviour on a dropped message
Delta	"buy 5"	Position drifts permanently
Target position	"net position should be −10"	Next message self-corrects

Where the account API supported it, I moved to a **target-position model**. The adapter keeps a running net position per instrument and transmits the *desired end state* rather than the increment.

This makes execution **idempotent**. A duplicated message is harmless. A dropped message is corrected by the next one. This is the layer where fire-and-forget transport gets its reliability back — and it belongs here, not in the transport, because only the adapter knows what the true target state is.

For accounts without that capability, a periodic reconciliation loop compares intended position against the account and closes any gap — the same idea implemented as a background sweep.

Fill verification

The most developed adapter does not treat submission as completion:

```
submit order
   └─ poll order history at a fixed interval
        ├─ terminal state reached  → log and return
        ├─ not filled              → retry with bounded backoff
        └─ deadline exceeded       → escalate
```

Retries are bounded and backed off. This closes the gap between "*the API accepted my order*" and "*my order actually filled*" — two very different facts that are easy to conflate.

---

8. Concurrency model

Two different concurrency models, chosen per layer:

Layer	Model	Reason
Strategy	Synchronous loop, one per process	Predictable timing; OS handles parallelism; no async complexity in the part of the code that changes most often
Execution	`asyncio` event loop	Adapters are I/O-bound — mostly waiting on HTTP responses
Execution (blocking calls)	`ThreadPoolExecutor`	Third-party SDKs are synchronous; offloading keeps the event loop responsive
Logging	Background thread + queue	Disk I/O never blocks a trading loop

The logging pattern is small but load-bearing. A trading loop calls `log(...)`, which only enqueues. A worker thread owns the file handle and does the write and flush.

```
trading loop ──enqueue──→ [ queue ] ──→ worker thread ──→ disk
   (never blocks)                          (owns the file)
```

The trading loop's timing is never coupled to disk latency. In a system where entries are timed to the second, that guarantee is worth the extra thread.

In the execution layer, each inbound message becomes an **independent async task**. One slow order does not delay the next one. Shared mutable state — sequence counters, deduplication sets — is protected by `asyncio.Lock` to make check-and-act sequences atomic rather than merely fast.

---

9. Observability and persistence

Three independent write paths, deliberately not consolidated:

Strategy-side trade log. Each strategy writes its own session CSV with a reason code on every action. This is the dataset for post-session analysis and attribution — each strategy's own account of what it did and why.

Execution-side order log. Each adapter appends every order it processed, timestamped to the microsecond. This is the record of what was actually *sent*.

Keeping these separate is intentional. When the two disagree, the gap between them **is** the diagnostic — it localises the problem to the transport or execution boundary immediately, rather than leaving you to guess.

Live position monitor. A dedicated process subscribes to the same signal bus but places no orders at all. It maintains the net position book and pushes updates to a messaging channel on change. Because it is a pure subscriber, it is completely non-invasive — it cannot affect execution, and it can be restarted at any moment without consequence.

That is the underrated benefit of a publish/subscribe bus: **monitoring is just another subscriber.** No hooks, no callbacks, no instrumentation inside the trading path.

---

10. Deployment

The system runs on a single cloud instance. Startup is ordered by dependency:

```
1.  Ingestion process         ← must populate the tick bus first
2.  Execution adapters        ← must be subscribed before signals fire
3.  Monitoring process
4.  Strategy processes        ← started before the session opens
```

Each process launches in its own labelled terminal tab, so at a glance you can see every component and its live output. It is deliberately low-tech, and for a single-instance system it has been more useful than a dashboard would have been — the state of every process is visible without querying anything.

Because each process is independent, any one of them can be restarted mid-session without disturbing the others.

---

11. Technology stack

Concern	Choice	Why
Language	Python 3.12 / 3.13	Numerical ecosystem; process isolation removes the GIL as a constraint
Tick transport	`mmap` + `struct`	Zero-copy shared memory; microsecond reads
Signal transport	ZeroMQ (PUB/SUB)	Non-blocking fan-out; brokerless; topic filtering at the socket
Serialisation	msgpack	Compact and fast on a hot path
Async I/O	`asyncio`	Execution layer is I/O-bound
Blocking-call bridge	`ThreadPoolExecutor`	Synchronous vendor SDKs
Market data	WebSocket clients	Continuous streaming
Numerics	NumPy, SciPy, pandas	Vectorised computation and analysis
Logging	Threaded queue writer	Disk I/O never blocks a trading loop

Two Python versions in one system is worth calling out again. One component is pinned to an older interpreter because of a third-party SDK; everything else runs on the newer one. Process isolation makes that a non-event rather than a codebase-wide constraint.

12. Scale

Concurrent processes	~40
Independent strategy processes	30+
Execution accounts served	8
Instruments in the live tick bus	~1,000
Dedicated signal-bus ports	21
Python source files	63
Lines of code	~19,500
Tick read latency	microsecond range
Ingestion connections required	1

That last row is the one that summarises the whole design: **one connection, thirty consumers, constant cost.**

---

13. What I would tell someone building this

Five things generalised beyond trading systems:

Process boundaries are the cheapest isolation you can buy. Threads share failure. Processes do not. If two components must not be able to break each other, the OS already provides the strongest guarantee available — and it is free.

Match the transport to the data's semantics. Ticks are *state* — the newest value invalidates the previous one, so shared memory is the natural fit. Orders are *events* — each one matters exactly once, so a message bus is the natural fit. Using one mechanism for both would have made one of them wrong.

Put reliability at the layer that has the information. The transport cannot know what "correct" means. The execution layer knows the intended position, so idempotency

belongs there. Pushing guarantees down to a layer that lacks the context to enforce them produces complexity without safety.

Configuration over code for anything operational. Which strategy feeds which account, and at what size, is a config file. Nothing about capital allocation should require a deployment.

Design so that monitoring is a subscriber, not a hook. If observing the system requires modifying the system, you will observe less than you should.

---

14. Closing

The interesting problem here was never any individual trading strategy. It was the **substrate**: how do you build a platform where thirty independent components share one expensive resource, never interfere with each other, and can each fail without consequence?

The answer turned out to be an old one. Pick the right primitive for each kind of data — shared memory for state, message bus for events — and let process boundaries do the isolation work that application code is bad at.

Happy to talk shop with anyone working on low-latency systems, IPC design, or multi-process architectures. The specific trading logic stays private, but the engineering is worth discussing — that part I learned from other people writing about their systems, so it seems only fair.

Note on scope. This document describes system architecture only. Trading logic, parameters, venue relationships and performance data are intentionally excluded — the engineering is the transferable part, and that is what is shared here.